

OBJECT ORIENTED ANALYSIS AND DESIGN

UNIT – I

Introduction to UML: Importance of modeling, principles of modeling, object oriented modeling, Conceptual model of the UML, Architecture, and Software Development Life Cycle.

UNIT – I

The UML is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML gives you a standard way to write a system's blueprints, covering conceptual things, such as business processes and system functions, as well as concrete things, such as classes written in a specific programming language, database schemas, and reusable software components.

Importance of modeling

A model is a simplification of reality. A model provides the blueprints of a system. A model may be structural, emphasizing the organization of the system, or it may be behavioral, emphasizing the dynamics of the system.

Aims of modeling

We build models so that we can better understand the system we are developing.

Through modeling, we achieve **four** aims.

1. Models help us to visualize a system as it is or as we want it to be.
2. Models permit us to specify the structure or behavior of a system.
3. Models give us a template that guides us in constructing a system.
4. Models document the decisions we have made.

We build models of complex systems because we cannot comprehend such a system in its entirety.

Principles of Modeling

There are **four** basic principles of model

1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.
2. Every model may be expressed at different levels of precision.
3. The best models are connected to reality.
4. No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models.

Object Oriented Modeling

In software, there are several ways to approach a model. The **two** most common ways are

1. Algorithmic perspective
2. Object-oriented perspective

Algorithmic Perspective

- The traditional view of software development takes an algorithmic perspective.
- In this approach, the main building block of all software is the procedure or

function.

- This view leads developers to focus on issues of control and the decomposition of larger algorithms into smaller ones.
- As requirements change and the system grows, systems built with an algorithmic focus turn out to be very hard to maintain.

Object-Oriented Perspective

- The contemporary view of software development takes an object-oriented perspective.
- In this approach, the main building block of all software systems is the object or class.
- A class is a description of a set of common objects.
- Every object has identity, state, and behavior.
- Object-oriented development provides the conceptual foundation for assembling systems out of components using technology such as Java Beans or COM+.

An Overview of UML

- The Unified Modeling Language is a standard language for writing software blueprints. The UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system.
- The UML is appropriate for modeling systems ranging from enterprise information systems to distributed Web-based applications and even to hard real time embedded systems. It is a very expressive language, addressing all the views needed to develop and then deploy such systems.

The UML is a language for

- Visualizing
 - Specifying
 - Constructing
 - Documenting
- **Visualizing** The UML is more than just a bunch of graphical symbols. Rather, behind each symbol in the UML notation is a well-defined semantics. In this manner, one developer can write a model in the UML, and another developer, or even another tool, can interpret that model unambiguously
- **Specifying** means building models that are precise, unambiguous, and complete.
- **Constructing** the UML is not a visual programming language, but its models can be directly connected to a variety of programming languages
- **Documenting** a healthy software organization produces all sorts of artifacts in addition to raw executable code. These artifacts include
 - Requirements
 - Architecture
 - Design
 - Source code
 - Project plans
 - Tests
 - Prototypes
 - Releases

Conceptual model of UML

To understand the UML, you need to form a **conceptual model of the language**, and this requires learning three major elements:

- **Basic building blocks of the UML**
- **Rules**
- **Common Mechanisms in the UML**

Basic building blocks of the UML: Vocabulary of the UML can be defined

1. Things
2. Relationships
3. Diagrams

Things in the UML

There are **four** kinds of things in the UML:

- Structural things
- Behavioral things
- Grouping things
- Annotational things

Structural things are the nouns of UML models. These are the mostly static parts of a model, representing elements that are either conceptual or physical. In all, there are **seven** kinds of structural things.

1. Classes
2. Interfaces
3. Collaborations
4. Use cases
5. Active classes
6. Components
7. Nodes

Class

- It is a description of a set of objects that share the same attributes, operations, relationships, and semantics.
- A class implements one or more interfaces.
- Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations.

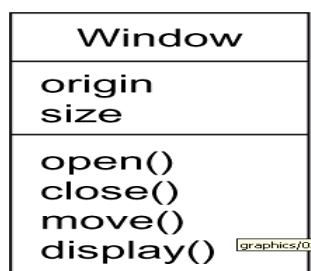


Fig: Class

Interface

- Interface is a collection of operations that specify a service of a class or component.
- An interface therefore describes the externally visible behavior of that element.
- An interface might represent the complete behavior of a class or component or only a part of that behavior.
- An interface is rendered as a circle together with its name. An interface rarely stands alone. Rather, it is typically attached to the class or component that realizes the interface.



Fig: Interface

Collaboration

- It defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements.
- Therefore, collaborations have structural, as well as behavioral, dimensions.
- A given class might participate in several collaborations.
- Graphically, a collaboration is rendered as an ellipse with dashed lines, usually including only its name.



Fig: Collaboration

Use case

- Use case is a description of set of sequence of actions that a system performs that yields an observable result of value to a particular actor
- Use case is used to structure the behavioral things in a model.
- A use case is realized by a collaboration. Graphically, a use case is rendered as an ellipse with solid lines, usually including only its name

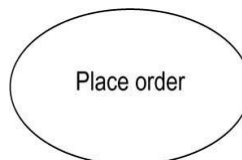


Fig: Use case

Active class

- It is just like a class except that its objects represent elements whose behavior is concurrent with other elements.
- Graphically, an active class is rendered just like a class, but with heavy lines, usually including its name, attributes, and operations.

- State machine involves a number of other elements, including states, transitions, events and activities
- Graphically, a state is rendered as a rounded rectangle, usually including its name and its substates

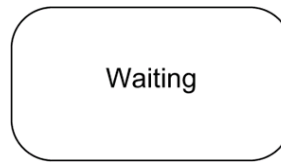


Fig: State machine

Grouping Things

1. These are the organizational parts of UML models. These are the boxes into which a model can be decomposed
2. There is **one** primary kind of grouping thing, namely, packages.

Package

- A package is a general-purpose mechanism for organizing elements into groups. Structural things, behavioral things, and even other grouping things may be placed in a package
- Graphically, a package is rendered as a tabbed folder, usually including only its name and, sometimes, its contents.



Fig: Package

Annotational things

- These are the explanatory parts of UML models. These are the comments you may apply to describe about any element in a model.
- There is one primary kind of Annotational thing, namely, note.

Note

- A note is simply a symbol for rendering constraints and comments attached to an element or a collection of elements.
- Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment.

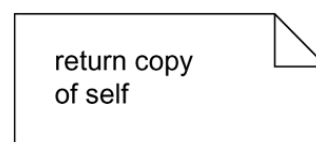


Fig: Note

Relationships in the UML: There are **four** kinds of relationships in the UML:

1. Dependency
2. Association
3. Generalization
4. Realization

Dependency

- Dependency is a semantic relationship between two things in which a change to one thing may affect the semantics of the other thing
- Graphically a dependency is rendered as a dashed line, possibly directed, and occasionally including a label.

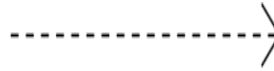


Fig: Dependency

Association

- Association is a structural relationship that describes a set of links, a link being a connection among objects.
- Graphically an association is rendered as a solid line, possibly directed, occasionally including a label, and often containing other adornments, such as multiplicity and role names.

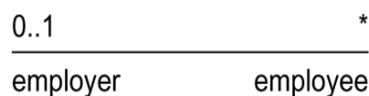


Fig: Association

Generalization

Generalization is a relationship that implements the concept of inheritance, a fundamental principle of object-oriented programming. This relationship occurs between two entities or objects, where one entity is the parent (or superclass) and the other is the child (or subclass). The child inherits the attributes, operations, and relationships of the parent, allowing for reuse and extension of existing functionality.



Fig: Generalization

Realization

- Realization is a semantic relationship between classifiers, wherein one classifier specifies a contract that another classifier guarantees to carry out.
- Graphically a realization relationship is rendered as a cross between a generalization and a dependency relationship.

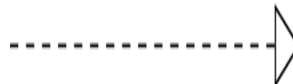


Fig: Realization

Diagrams in the UML

- Diagram is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).
- In theory, a diagram may contain any combination of things and relationships.
- For this reason, the UML includes **nine** such diagrams

- Class diagram
- Object diagram
- Use case diagram
- Sequence diagram
- Collaboration diagram
- State chart diagram
- Activity diagram
- Component diagram
- Deployment diagram

Class diagram

- A class diagram shows a set of classes, interfaces, and collaborations and their relationships.
- Class diagrams that include active classes address the static process view of a system.

Object diagram

- Object diagrams represent static snapshots of instances of the things found in class diagrams
- These diagrams address the static design view or static process view of a system
- An object diagram shows a set of objects and their relationships

Use case diagram

- A use case diagram shows a set of use cases and actors and their relationships
- Use case diagrams address the static use case view of a system.
- These diagrams are especially important in organizing and modeling the behaviors of a system.

Interaction Diagrams

- Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams
- Interaction diagrams address the dynamic view of a system
- **A sequence diagram** is an interaction diagram that emphasizes the time-ordering of messages
- **A collaboration diagram** is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages
- Sequence diagrams and collaboration diagrams are **isomorphic**, meaning that you can take one and transform it into the other.

Statechart diagram

- A state chart diagram shows a state machine, consisting of states, transitions, events, and activities
- State chart diagrams address the dynamic view of a system
- They are especially important in modeling the behavior of an interface, class, or collaboration and emphasize the event-ordered behavior of an object.

Activity diagram

An activity diagram is a special kind of a state chart diagram that shows the flow from activity to activity within a system

Activity diagrams address the dynamic view of a system

They are especially important in modeling the function of a system and emphasize the flow of control among objects.

Component diagram

- A component diagram shows the organizations and dependencies among a set of components.
- Component diagrams address the static implementation view of a system
- They are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations.

Deployment diagram

- A deployment diagram shows the configuration of run-time processing nodes and the components that live on them
- Deployment diagrams address the static deployment view of an architecture

Rules of the UML

The UML has semantic rules for

- | | |
|---------------|--|
| 1. Names | What you can call things, relationships, and diagrams |
| 2. Scope | The context that gives specific meaning to a name |
| 3. Visibility | How those names can be seen and used by others |
| 4. Integrity | How things properly and consistently relate to one another |
| 5. Execution | What it means to run or simulate a dynamic model |

Models built during the development of a software-intensive system tend to evolve and may be viewed by many stakeholders in different ways and at different times. For this reason, it is common for the development team to not only build models that are well-formed, but also to build models that are

1. Elided Certain elements are hidden to simplify the view
2. Incomplete Certain elements may be missing
3. Inconsistent The integrity of the model is not guaranteed

Common Mechanisms in the UML

UML is made simpler by the presence of **four** common mechanisms that apply consistently throughout the language.

1. Specifications
2. Adornments
3. Common divisions
4. Extensibility mechanisms

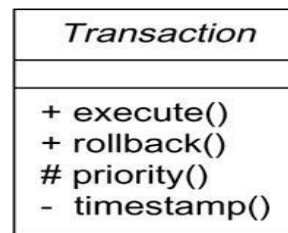
Specification

- They provides a textual statement of the syntax and semantics of that building block.
- The UML's specifications provide a semantic backplane that contains all the parts of all the models of a system, each part related to one another in a consistent fashion

Adornments

- Most elements in the UML have a unique and direct graphical notation that provides a visual representation of the most important aspects of the element.
- A class's specification may include other details, such as whether it is abstract or the visibility of its attributes and operations.

- Many of these details can be rendered as graphical or textual adornments to the class's basic rectangular notation.

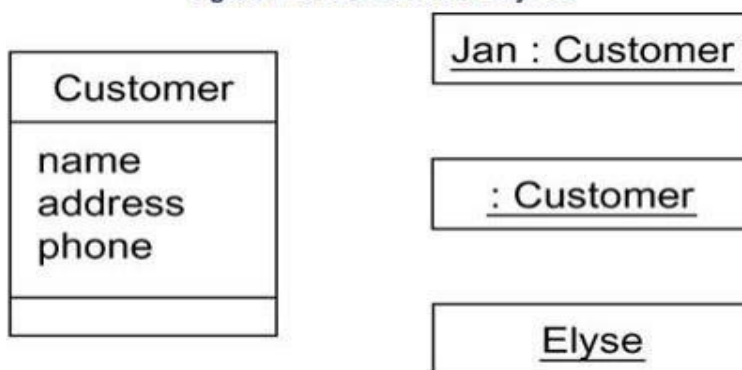


Common Divisions

In modeling object-oriented systems, the world often gets divided in at least a couple of ways.

First, there is the division of class and object. A class is an abstraction; an object is one concrete manifestation of that abstraction. In the UML, you can model classes as well as objects, as shown in Figure 2-17

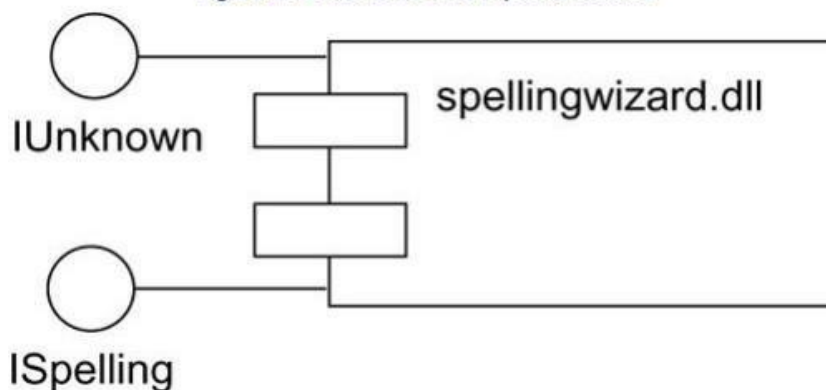
Figure 2-17 Classes And Objects



In this figure, there is one class, named *Customer*, together with three objects: *Jan* (which is marked explicitly as being a *Customer* object), *:Customer* (an anonymous *Customer* object), and *Elyse* (which in its specification is marked as being a kind of *Customer* object, although it's not shown explicitly here).

Second, there is the separation of interface and implementation. An interface declares a contract, and an implementation represents one concrete realization of that contract, responsible for faithfully carrying out the interface's complete semantics. In the UML, you can model both interfaces and their implementations, as shown in Figure 2-18.

Figure 2-18 Interfaces And Implementations



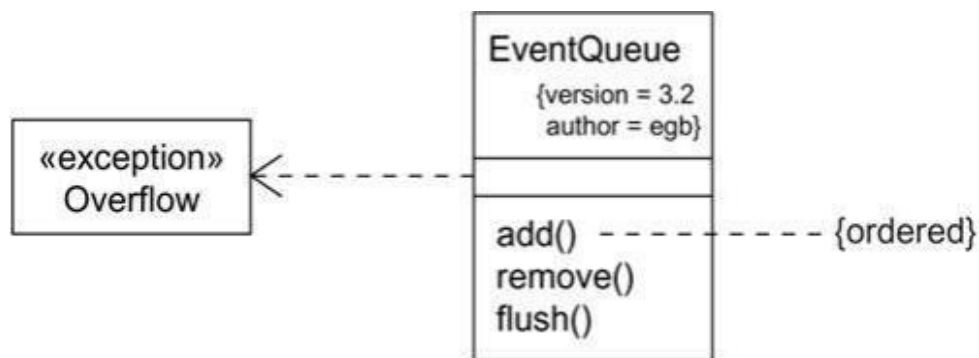
In this figure, there is one component named *spellingwizard.dll* that implements two interfaces, *IUnknown* and *ISpelling*.

Extensibility Mechanisms

The UML's extensibility mechanisms include

1. Stereotypes
2. Tagged values
3. Constraints

- **Stereotype** extends the vocabulary of the UML, allowing you to create new kinds of building blocks that are derived from existing ones but that are specific to your problem
- A **tagged value** extends the properties of a UML building block, allowing you to create new information in that element's specification
- A **constraint** extends the semantics of a UML building block, allowing you to add new rules or modify existing ones. Examples for these are



Architecture

- A system's architecture is perhaps the most important artifact that can be used to manage these different viewpoints and so control the iterative and incremental development of a system throughout its life cycle.
- Architecture is the set of significant decisions about
- The organization of a software system
- The selection of the structural elements and their interfaces by which the system is composed
- Their behavior, as specified in the collaborations among those elements
- The composition of these structural and behavioral elements into progressively larger subsystems
- The architectural style that guides this organization: the static and dynamic elements and their interfaces, their collaborations, and their composition.
- Software architecture is not only concerned with structure and behavior, but also with usage, functionality, performance, resilience, reuse, comprehensibility, economic and technology constraints and trade-offs, and aesthetic concerns.

Figure 2-20 Modeling a System's Architecture

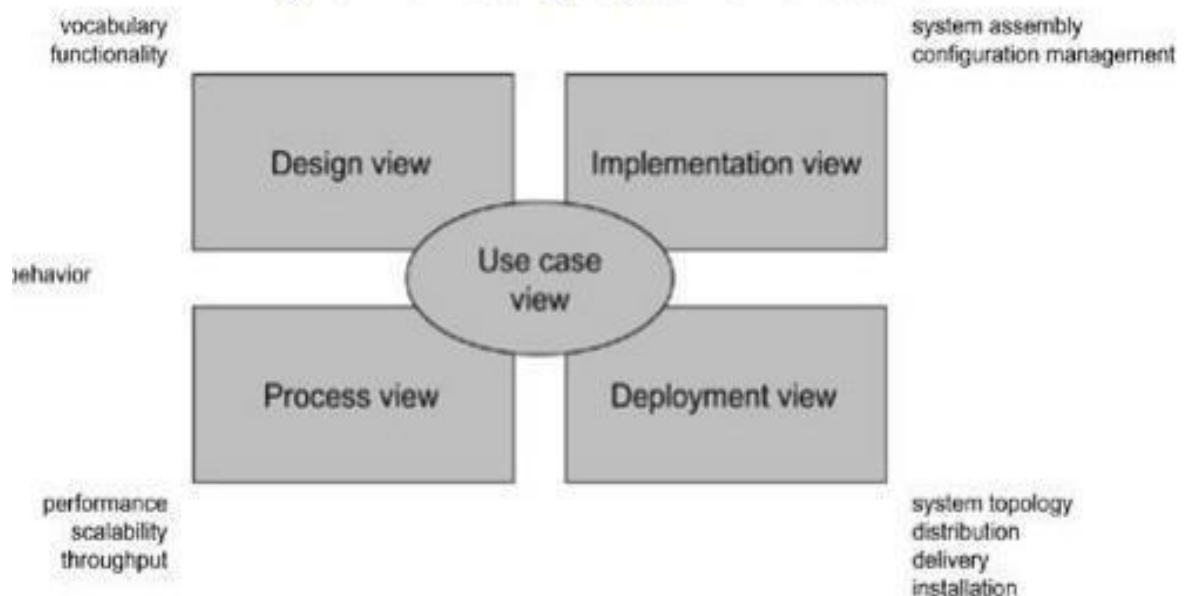


Fig: Modeling a System's Architecture

Use case view

- The use case view of a system encompasses the use cases that describe the behavior of the system as seen by its end users, analysts, and testers.
- With the UML, the static aspects of this view are captured in use case diagrams
- The dynamic aspects of this view are captured in interaction diagrams, state chart diagrams, and activity diagrams.

Design View

- The design view of a system encompasses the classes, interfaces, and collaborations that form the vocabulary of the problem and its solution.
- This view primarily supports the functional requirements of the system, meaning the services that the system should provide to its end users.

Process View

- The process view of a system encompasses the threads and processes that form the system's concurrency and synchronization mechanisms.
- This view primarily addresses the performance, scalability, and throughput of the system.

Implementation View

- The implementation view of a system encompasses the components and files that are used to assemble and release the physical system.
- This view primarily addresses the configuration management of the system's releases, made up of somewhat independent components and files that can be assembled in various ways to produce a running system.

Deployment View

- The deployment view of a system encompasses the nodes that form the system's hardware topology on which the system executes.
- This view primarily addresses the distribution, delivery, and installation of the parts that make up the physical system.

- With the UML, the static aspects of this view are captured in deployment diagrams; the dynamic aspects of this view are captured in interaction diagrams, statechart diagrams, and activity diagrams.
- Each of these five views can stand alone so that different stakeholders can focus on the issues of the system's architecture that most concern them.
- These five views also interact with one another• nodes in the deployment view hold components in the implementation view that, in turn, represent the physical realization of classes, interfaces, collaborations, and active classes from the design and process views.
- The UML permits you to express every one of these five views and their interactions.

Software Development Life Cycle

- The UML is largely process-independent, meaning that it is not tied to any particular software development life cycle.
- However, to get the most benefit from the UML, you should consider a process that is
 - Use case driven
 - Architecture-centric
 - Iterative and incremental

Use case driven

- It means that use cases are used as a primary artifact for establishing the desired behavior of the system, for verifying and validating the system's architecture, for testing, and for communicating among the stakeholders of the project.

Architecture-centric

- It means that a system's architecture is used as a primary artifact for Conceptualizing, constructing, managing, and evolving the system under development.

An iterative process

- It is one that involves managing a stream of executable releases.

An incremental process

- It is one that involves the continuous integration of the system's architecture to produce these releases, with each new release embodying incremental improvements over the other.
- Together, an iterative and incremental process is *risk-driven*, meaning that each new release is focused on attacking and reducing the most significant risks to the success of the project. This use case driven, architecture-centric, and iterative/incremental process can be broken into phases.
- **A phase** is the span of time between two major milestones of the process, when a well-defined set of objectives are met, artifacts are completed, and decisions are made whether to move into the next phase.
- As below figure shows, there are **four** phases in the software development life cycle: inception, elaboration, construction, and transition. In the diagram,

workflows are plotted against these phases, showing their varying degrees of focus over time.

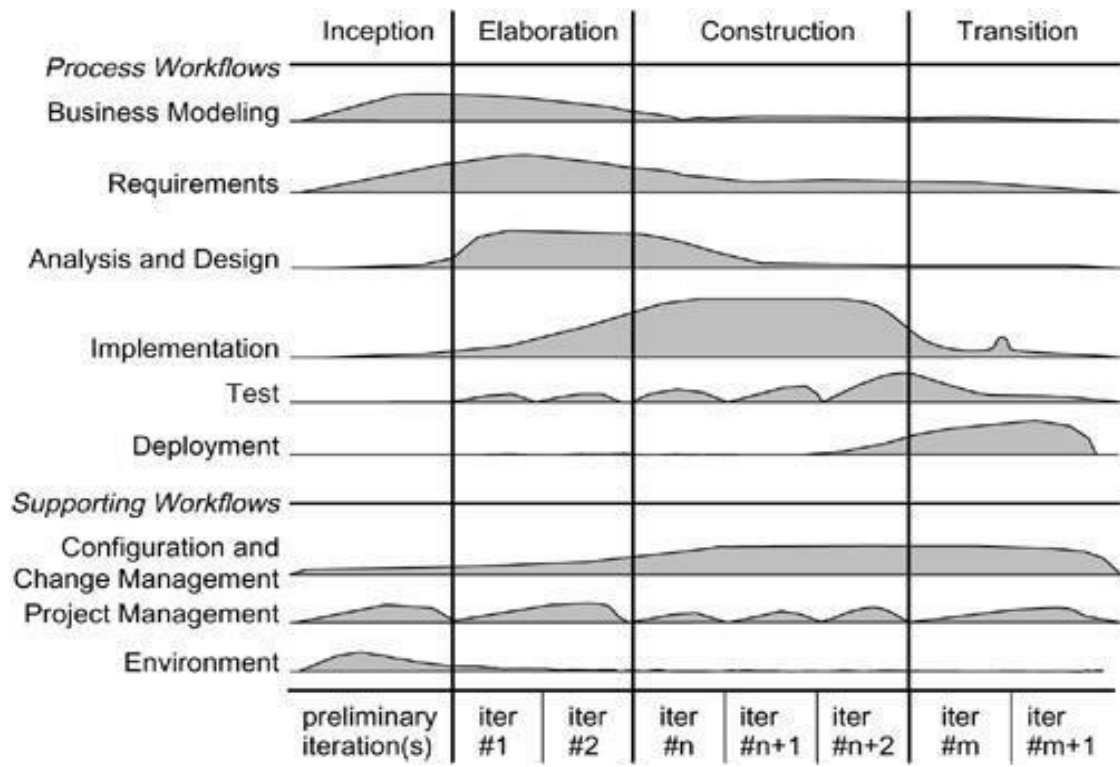


Fig: Figure Software Development Life Cycle

Inception phase:

- It is the first phase of the process
- When the seed idea for the development is brought up to the point of being at least internally sufficiently well-founded to warrant entering into the elaboration phase.

Elaboration phase:

- It is the second phase of the process
- When the product vision and its architecture are defined.
- In this phase, the system's requirements are articulated, prioritized, and baselined.
- A system's requirements may range from general vision statements to precise evaluation criteria, each specifying particular functional or nonfunctional behavior and each providing a basis for testing.

Construction phase:

- It is the third phase of the process.
- When the software is brought from an executable architectural baseline to being ready to be transitioned to the user community.
- The system's requirements and especially its evaluation criteria are constantly reexamined against the business needs of the project, and resources are allocated as appropriate to actively attack risks to the project.

Transition phase:

- It is the fourth phase of the process.
- When the software is turned into the hands of the user community.

- The software development process end here, for even during this phase, the system is continuously improved, bugs are eradicated, and features that didn't make an earlier release are added.
- One element that distinguishes this process and that cuts across all four phases is an iteration.
- An iteration is a distinct set of activities, with a baselined plan and evaluation criteria that result in a release, either internal or external.
- This means that the software development life cycle can be characterized as involving a continuous stream of executable releases of the system's architecture.

It is this emphasis on architecture as an important artifact that drives the UML to focus on modeling the different views of a

OBJECT ORIENTED ANALYSIS AND DESIGN

UNIT – I

Introduction to UML: Importance of modeling, principles of modeling, object oriented modeling, Conceptual model of the UML, Architecture, and Software Development Life Cycle.

UNIT – I

The UML is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML gives you a standard way to write a system's blueprints, covering conceptual things, such as business processes and system functions, as well as concrete things, such as classes written in a specific programming language, database schemas, and reusable software components.

Importance of modeling

A model is a simplification of reality. A model provides the blueprints of a system. A model may be structural, emphasizing the organization of the system, or it may be behavioral, emphasizing the dynamics of the system.

Aims of modeling

We build models so that we can better understand the system we are developing.

Through modeling, we achieve **four** aims.

1. Models help us to visualize a system as it is or as we want it to be.
2. Models permit us to specify the structure or behavior of a system.
3. Models give us a template that guides us in constructing a system.
4. Models document the decisions we have made.

We build models of complex systems because we cannot comprehend such a system in its entirety.

Principles of Modeling

There are **four** basic principles of model

1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.
2. Every model may be expressed at different levels of precision.
3. The best models are connected to reality.
4. No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models.

Object Oriented Modeling

In software, there are several ways to approach a model. The **two** most common ways are

1. Algorithmic perspective
2. Object-oriented perspective

Algorithmic Perspective

- The traditional view of software development takes an algorithmic perspective.
- In this approach, the main building block of all software is the procedure or